

Solucionario Técnico: Bloque 3

Ecosistema XML: Modelos DOM, Transformaciones XSLT, Parsers SAX/StAX y Mapeo POJO JAXB

MÓDULO PROFESIONAL	CICLO Y UNIDAD	MODALIDAD / ENTORNO	DESTINATARIO / CARÁCTER
Acceso a Datos (0486)	DAM 2º Curso · UT1	Presencial / Distancia · Java 21	Docente · Solución Oficial Comentada

ALINEACIÓN CURRICULAR OFICIAL (REAL DECRETO 455/2010 · CURRÍCULO ASTURIAS)

RA1: Manejo de Ficheros

CE1.e (Formatos de Intercambio de Información: XML)

Solucionario oficial que articula la creación y modificación de árboles jerárquicos DOM con `Transformer`, el procesamiento reactivo de eventos streaming con `SAXParser` y `DefaultHandler`, el control iterativo Pull con `XMLStreamReader` (StAX) y el intercambio bidireccional objeto-relacional declarativo mediante Jakarta XML Binding (JAXB).

Solución Ejercicio 1: Generador de Facturas DOM con Transformer

(FacturaDOM.java)

NIVEL
BÁSICO

Claves Técnicas de la Implementación:

- **Construcción Jerárquica:** Creación ordenada con `doc.createElement()`, asignación de atributos con `setAttribute()` y adición de nodos de texto con `setTextContent()`.
- **Pretty-Print con Transformer:** Se configuran las propiedades `OutputKeys.INDENT = "yes"` y la propiedad Apache `{http://xml.apache.org/xslt}indent-amount = "4"` para volcar el árbol con sangría profesional.

Código Fuente Completo:

```

package soluciones_b3;

import org.w3c.dom.Document;
import org.w3c.dom.Element;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import java.io.File;

public class FacturaDOM {

    public static void main(String[] args) {
        try {
            DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
            DocumentBuilder builder = factory.newDocumentBuilder();
            Document doc = builder.newDocument();

            // Elemento raíz:
            Element raiz = doc.createElement("factura");
            raiz.setAttribute("numero", "FAC-2026-0042");
            raiz.setAttribute("fecha", "2026-09-17");
            doc.appendChild(raiz);

            //
            Element emisor = doc.createElement("emisor");
            emisor.setAttribute("cif", "B-33123456");
            Element nomEmisor = doc.createElement("nombre");
            nomEmisor.setTextContent("ElectroTech S.L.");
            Element ciuEmisor = doc.createElement("ciudad");
            ciuEmisor.setTextContent("Avilés");
            emisor.appendChild(nomEmisor);
            emisor.appendChild(ciuEmisor);
            raiz.appendChild(emisor);

            //
            Element cliente = doc.createElement("cliente");
            cliente.setAttribute("nif", "71888999X");
            Element nomCliente = doc.createElement("nombre");
            nomCliente.setTextContent("Desarrollos Astur S.A.");
            Element ciuCliente = doc.createElement("ciudad");
            ciuCliente.setTextContent("Gijón");
            cliente.appendChild(nomCliente);
            cliente.appendChild(ciuCliente);
            raiz.appendChild(cliente);

            //
            Element lineas = doc.createElement("lineas");
            raiz.appendChild(lineas);

            // Línea 1
            Element l1 = doc.createElement("linea");
            l1.setAttribute("id", "1");
            Element c1 = doc.createElement("concepto");
            c1.setTextContent("Servidor Rack 1U");

```

```

        Element q1 = doc.createElement("cantidad");
        q1.setTextContent("2");
        Element p1 = doc.createElement("precioUnitario");
        p1.setTextContent("850.00");
        l1.appendChild(c1);
        l1.appendChild(q1);
        l1.appendChild(p1);
        lineas.appendChild(l1);

        // Línea 2
        Element l2 = doc.createElement("linea");
        l2.setAttribute("id", "2");
        Element c2 = doc.createElement("concepto");
        c2.setTextContent("Switch Gigabit 24 Puertos");
        Element q2 = doc.createElement("cantidad");
        q2.setTextContent("1");
        Element p2 = doc.createElement("precioUnitario");
        p2.setTextContent("145.50");
        l2.appendChild(c2);
        l2.appendChild(q2);
        l2.appendChild(p2);
        lineas.appendChild(l2);

        // Serialización a disco con sangría
        TransformerFactory tfFactory = TransformerFactory.newInstance();
        Transformer tf = tfFactory.newTransformer();
        tf.setOutputProperty(OutputKeys.INDENT, "yes");
        tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");
        tf.setOutputProperty(OutputKeys.ENCODING, "UTF-8");

        File destino = new File("datos/factura_electronica.xml");
        destino.getParentFile().mkdirs();
        tf.transform(new DOMSource(doc), new StreamResult(destino));

        System.out.println("✅ Factura XML generada e indentada en: " + destino.getPath());

    } catch (Exception e) {
        System.err.println("Error generando factura DOM: " + e.getMessage());
    }
}
}

```

Solución Ejercicio 2: Modificador de Tarifas DOM In-Situ

(GestorTarifasDOM.java)

NIVEL
INTERMEDIO

Claves Técnicas de la Implementación:

- **Normalización Previa:** `doc.getDocumentElement().normalize()` combina fragmentos de texto adyacentes producidos tras comentarios o entidades.
- **Filtro Anti-#text:** Comprobación sistemática `if (nodo.getNodeType() != Node.ELEMENT_NODE) continue;` para descartar nodos de espacio en blanco.
- **Sobrescritura In-Situ:** Se reemplaza el contenido del nodo con `elemPrecio.setTextContent(...)` y se vuelve a volcar el documento sobre el mismo fichero con `Transformer`.



Código Fuente Completo:

```

package soluciones_b3;

import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;

import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import java.io.File;
import java.util.Locale;

public class GestorTarifasDOM {

    public static void aplicarDescuentoCategoria(File archivo, String categoriaObjetivo, double porcentaje) {
        try {
            Document doc = DocumentBuilderFactory.newInstance()
                .newDocumentBuilder()
                .parse(archivo);

            doc.getDocumentElement().normalize();

            NodeList productos = doc.getElementsByTagName("producto");
            int modificados = 0;

            System.out.println("=====");
            System.out.printf("    APLICANDO %.1f%% DE DESCUENTO A CATEGORÍA: %s\n", porcentaje, categoriaObjetivo);
            System.out.println("=====");

            for (int i = 0; i < productos.getLength(); i++) {
                Node nodo = productos.item(i);
                if (nodo.getNodeType() != Node.ELEMENT_NODE) continue;

                Element elemProducto = (Element) nodo;
                String cat = elemProducto.getAttribute("categoria");

                if (categoriaObjetivo.equalsIgnoreCase(cat)) {
                    NodeList hijos = elemProducto.getElementsByTagName("precio");
                    if (hijos.getLength() > 0) {
                        Element elemPrecio = (Element) hijos.item(0);
                        double precioAntiguo = Double.parseDouble(elemPrecio.getTextContent().trim());
                        double nuevoPrecio = precioAntiguo * (1.0 - (porcentaje / 100.0));

                        elemPrecio.setTextContent(String.format(Locale.US, "%.2f", nuevoPrecio));

                        String nombre = elemProducto.getElementsByTagName("nombre").item(0).getTextContent();
                        System.out.printf("    - %-28s : %6.2f € -> %6.2f €\n", nombre, precioAntiguo, nuevoPrecio);
                        modificados++;
                    }
                }
            }

            Transformer tf = TransformerFactory.newInstance().newTransformer();
            tf.setOutputProperty(OutputKeys.INDENT, "yes");
            tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

```
tf.transform(new DOMSource(doc), new StreamResult(archivo));

System.out.println("-----");
System.out.printf("✅ Tarifa actualizada en disco: %d productos modificados.%n", modifi

} catch (Exception e) {
    System.err.println("Error actualizando tarifas DOM: " + e.getMessage());
}

}

public static void main(String[] args) {
    File f = new File("datos/catalogo_tienda.xml");
    aplicarDescuentoCategoria(f, "Perifericos", 15.0);
}

}
```

Solución Ejercicio 3: Analizador Dual Streaming SAX y StAX

(`AnalizadorDualXML.java`)

NIVEL
AVANZADO

Claves Técnicas de la Implementación:

- **Acumulador SAX en `characters()`** : `buffer.append(ch, start, length)` previene pérdidas de datos cuando el parser fragmenta textos extensos en varios paquetes.
- **StAX Pull con Salida Inmediata:** En cuanto se detecta la etiqueta buscada y coincide el atributo `id`, se leen los datos y se ejecuta `break`, evitando procesar el resto de un fichero de 500 MB.

Código Fuente Completo:

```

package soluciones_b3;

import org.xml.sax.Attributes;
import org.xml.sax.helpers.DefaultHandler;

import javax.xml.parsers.SAXParserFactory;
import javax.xml.stream.XMLInputFactory;
import javax.xml.stream.XMLStreamConstants;
import javax.xml.stream.XMLStreamReader;
import java.io.File;
import java.io.FileReader;
import java.util.Locale;

public class AnalizadorDualXML {

    // PARTE A: SAX PUSH
    public static class CatalogoStatsHandler extends DefaultHandler {
        private final StringBuilder buffer = new StringBuilder();
        private int totalProductos = 0;
        private double valorTotalInventario = 0;
        private double precioActual = 0;
        private int stockActual = 0;
        private String nombreMasCaro = "";
        private double precioMasCaro = -1;
        private String nombreActual = "";

        @Override
        public void startElement(String uri, String localName, String qName, Attributes attributes) {
            buffer.setLength(0);
            if (qName.equalsIgnoreCase("producto")) {
                totalProductos++;
            }
        }

        @Override
        public void characters(char[] ch, int start, int length) {
            buffer.append(ch, start, length);
        }

        @Override
        public void endElement(String uri, String localName, String qName) {
            String texto = buffer.toString().trim();
            if (qName.equalsIgnoreCase("nombre")) {
                nombreActual = texto;
            } else if (qName.equalsIgnoreCase("precio")) {
                precioActual = Double.parseDouble(texto);
                if (precioActual > precioMasCaro) {
                    precioMasCaro = precioActual;
                    nombreMasCaro = nombreActual;
                }
            } else if (qName.equalsIgnoreCase("stock")) {
                stockActual = Integer.parseInt(texto);
                valorTotalInventario += (precioActual * stockActual);
            }
        }

        @Override
        public void endDocument() {
            System.out.println("=====");
        }
    }
}

```

```

        System.out.println("                INFORME ESTADÍSTICO SAX (CONSUMO RAM: ~0 KB)                ");
        System.out.println("=====");
        System.out.printf(Locale.US, "1. Total productos analizados: %d\n", totalProductos);
        System.out.printf(Locale.US, "2. Valor total del inventario: %,2f €\n", valorTotalInventario);
        System.out.printf(Locale.US, "3. Producto más caro: %s (%,2f €)\n", nombreMasCaro, precioMasCaro);
        System.out.println("=====");
    }
}

public static void ejecutarSAX(File archivo) {
    try {
        SAXParserFactory.newInstance().newSAXParser().parse(archivo, new CatalogoStatsHandler());
    } catch (Exception e) {
        System.err.println("Error en parseo SAX: " + e.getMessage());
    }
}

// PARTE B: StAX PULL
public static void buscarProductoPorIdStAX(File archivo, String idBuscado) {
    System.out.println("\n--- Búsqueda Directa con StAX Cursor (Early Exit) ---");
    try (FileReader fr = new FileReader(archivo)) {
        XMLStreamReader reader = XMLInputFactory.newInstance().createXMLStreamReader(fr);

        boolean enProductoObjetivo = false;
        String nombre = "";
        String precio = "";
        String stock = "";

        while (reader.hasNext()) {
            int evento = reader.next();

            if (evento == XMLStreamConstants.START_ELEMENT) {
                String tag = reader.getLocalName();
                if ("producto".equals(tag)) {
                    String id = reader.getAttributeValue(null, "id");
                    if (idBuscado.equalsIgnoreCase(id)) enProductoObjetivo = true;
                } else if (enProductoObjetivo) {
                    if ("nombre".equals(tag)) nombre = reader.getElementText();
                    else if ("precio".equals(tag)) precio = reader.getElementText();
                    else if ("stock".equals(tag)) stock = reader.getElementText();
                }
            } else if (evento == XMLStreamConstants.END_ELEMENT) {
                if ("producto".equals(reader.getLocalName()) && enProductoObjetivo) {
                    System.out.printf("🔍 Encontrado con StAX: [%s] %s | Precio: %s € | Stock: %s\n",
                        idBuscado, nombre, precio, stock);
                    System.out.println("⚡ Bucle StAX detenido con 'break' sin leer el resto de XML");
                    break;
                }
            }
        }
    } catch (Exception e) {
        System.err.println("Error en búsqueda StAX: " + e.getMessage());
    }
}

public static void main(String[] args) {
    File f = new File("datos/catalogo_tienda.xml");
    ejecutarSAX(f);
    buscarProductoPorIdStAX(f, "P02");
}

```

```
}  
}
```

Solución Ejercicio 4: Mapeo Declarativo Objeto-XML con JAXB

(BibliotecaJAXB.java)

NIVEL
AVANZADO

Claves Técnicas de la Implementación:

- **Anotaciones Estándar Jakarta:** `@XmlRootElement` , `@XmlAccessorType(XmlAccessType.FIELD)` , `@XmlAttribute` y `@XmlElement` vinculan automáticamente los atributos de la clase con las etiquetas XML.
- **Constructores Vacíos:** Imprescindibles para que el motor de introspección de JAXB pueda recrear las instancias en tiempo de ejecución.
- **Marshaller y Formateo:** `m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true)` asegura que el XML se genere con saltos de línea e indentación legible.



Código Fuente Completo:

```

package soluciones_b3;

import jakarta.xml.bind.JAXBContext;
import jakarta.xml.bind.Marshaller;
import jakarta.xml.bind.Unmarshaller;
import jakarta.xml.bind.annotation.*;

import java.io.File;
import java.util.ArrayList;
import java.util.List;

public class BibliotecaJAXB {

    // 1. Clase Libro
    @XmlAccessorType(XmlAccessType.FIELD)
    public static class Libro {
        @XmlAttribute
        private String isbn;
        @XmlElement
        private String titulo;
        @XmlElement
        private String autor;
        @XmlElement
        private double precio;
        @XmlElement
        private int paginas;

        public Libro() {} // Constructor vacío obligatorio

        public Libro(String isbn, String titulo, String autor, double precio, int paginas) {
            this.isbn = isbn;
            this.titulo = titulo;
            this.autor = autor;
            this.precio = precio;
            this.paginas = paginas;
        }

        @Override
        public String toString() {
            return String.format(" * [%s] '%s' de %s (%d págs.) -> %.2f €",
                isbn, titulo, autor, paginas, precio);
        }
    }

    // 2. Clase Biblioteca (Raíz)
    @XmlRootElement(name = "biblioteca")
    @XmlAccessorType(XmlAccessType.FIELD)
    public static class Biblioteca {
        @XmlAttribute
        private String nombre;
        @XmlElement(name = "libro")
        private List libros = new ArrayList<>();

        public Biblioteca() {}

        public Biblioteca(String nombre) {
            this.nombre = nombre;
        }
    }
}

```

```

        public void agregarLibro(Libro l) {
            this.libros.add(l);
        }

        public List getLibros() {
            return libros;
        }
    }

    // 3. Operaciones Marshaller y Unmarshaller
    public static void exportarBiblioteca(Biblioteca b, File destino) throws Exception {
        JAXBContext context = JAXBContext.newInstance(Biblioteca.class);
        Marshaller m = context.createMarshaller();
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
        m.setProperty(Marshaller.JAXB_ENCODING, "UTF-8");

        destino.getParentFile().mkdirs();
        m.marshal(b, destino);
        System.out.println("✅ Biblioteca serializada a XML con JAXB en: " + destino.getPath());
    }

    public static Biblioteca importarBiblioteca(File origen) throws Exception {
        JAXBContext context = JAXBContext.newInstance(Biblioteca.class);
        Unmarshaller um = context.createUnmarshaller();
        return (Biblioteca) um.unmarshal(origen);
    }

    public static void main(String[] args) throws Exception {
        File f = new File("datos/biblioteca.xml");

        Biblioteca bib = new Biblioteca("Biblioteca Municipal de Avilés");
        bib.agregarLibro(new Libro("978-84-1", "Aprende Java 21 en FP", "Sergio Capdevila", 29.95, 4));
        bib.agregarLibro(new Libro("978-84-2", "Patrones de Acceso a Datos", "Martin Fowler", 42.50, 4));
        bib.agregarLibro(new Libro("978-84-3", "Arquitectura Limpia", "Robert C. Martin", 38.00, 380));

        exportarBiblioteca(bib, f);

        System.out.println("\n--- Recuperando desde XML con JAXB ---");
        Biblioteca recuperada = importarBiblioteca(f);
        System.out.println("Biblioteca cargada: " + recuperada.nombre);
        recuperada.getLibros().forEach(System.out::println);
    }
}

```